

Harness CI/CD: The AI Integration Strategy

Synergizing our pipelines for
Q3 and beyond.

Action Item:
Stop writing
YAML from
scratch.
- SRE Team

Why are we pivoting? (The Low-Hanging Fruit)

- Manual YAML wrangling & syntax errors
- Forgetting JPMC compliance (ServiceNow, SAML)
- Slow, painful debugging sessions



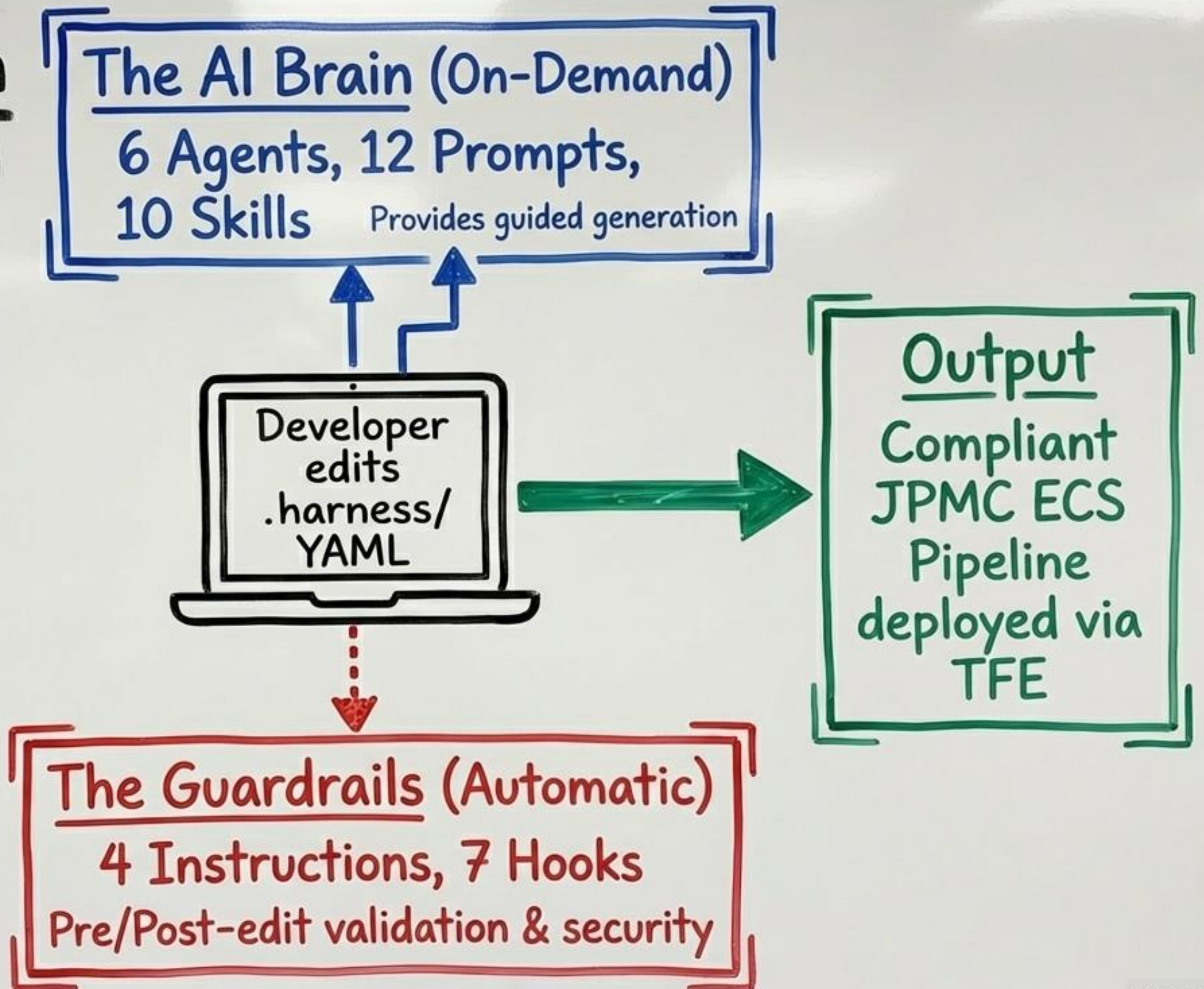
Value Add:
Less pain,
more shipping.

- AI-Assisted Copilot & Claude integration
- Automated enterprise guardrails & security scans
- Massive cost & speed optimization



The Ecosystem

(Let's unpack this)



Paradigm Alignment:

What tool do I actually use?

What's the ask?

I need step-by-step guidance for a known task.

Use a Prompt
e.g., create-ci-pipeline

I have a complex logic problem or multi-step workflow.

Use an Agent
e.g., @HarnessPipelineDeveloper

I just need to understand how a feature works.

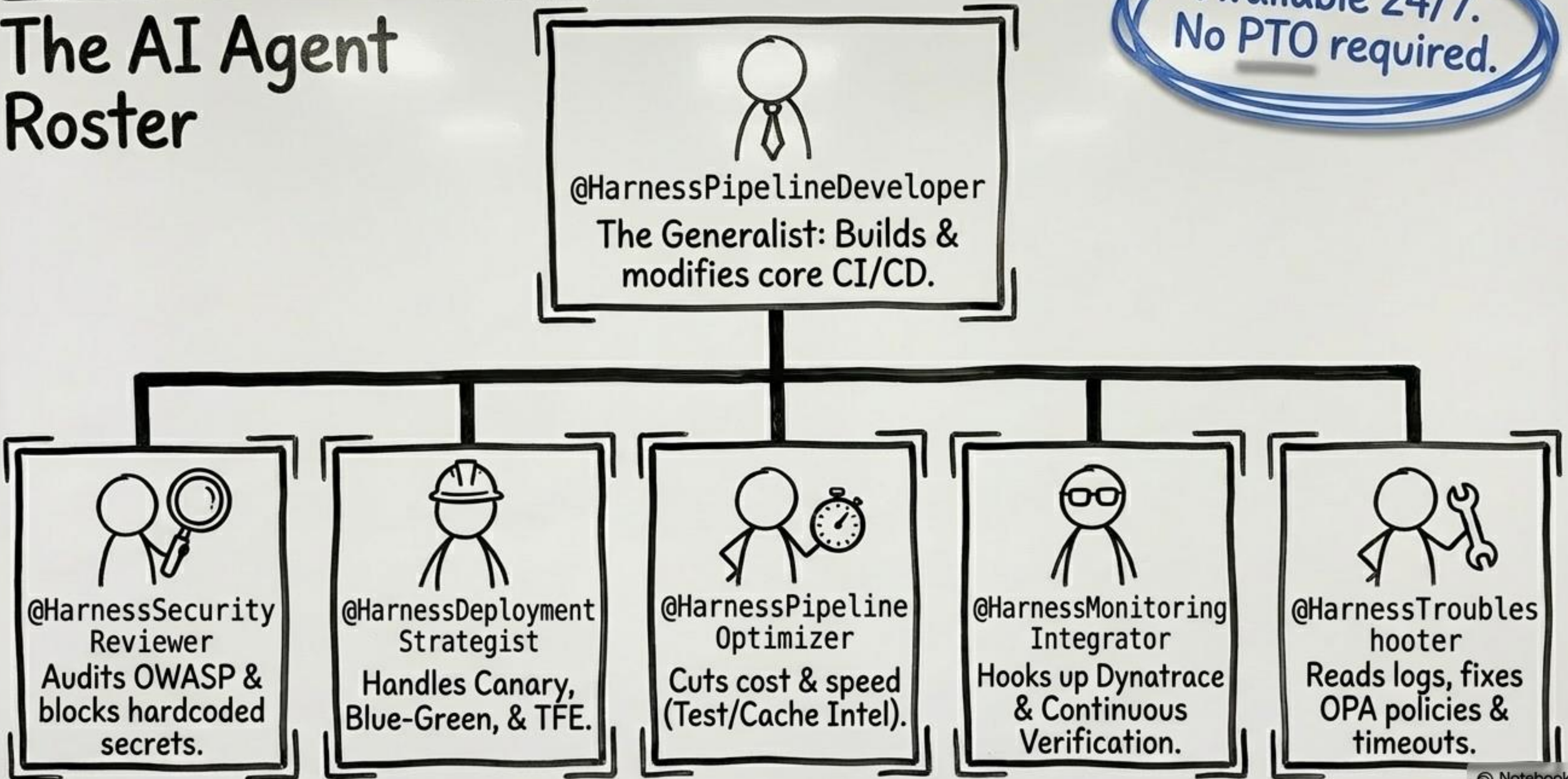
Use a Skill
e.g., read SKILL.md

*** Don't reinvent the wheel.
Always try to leverage the prompts first!**

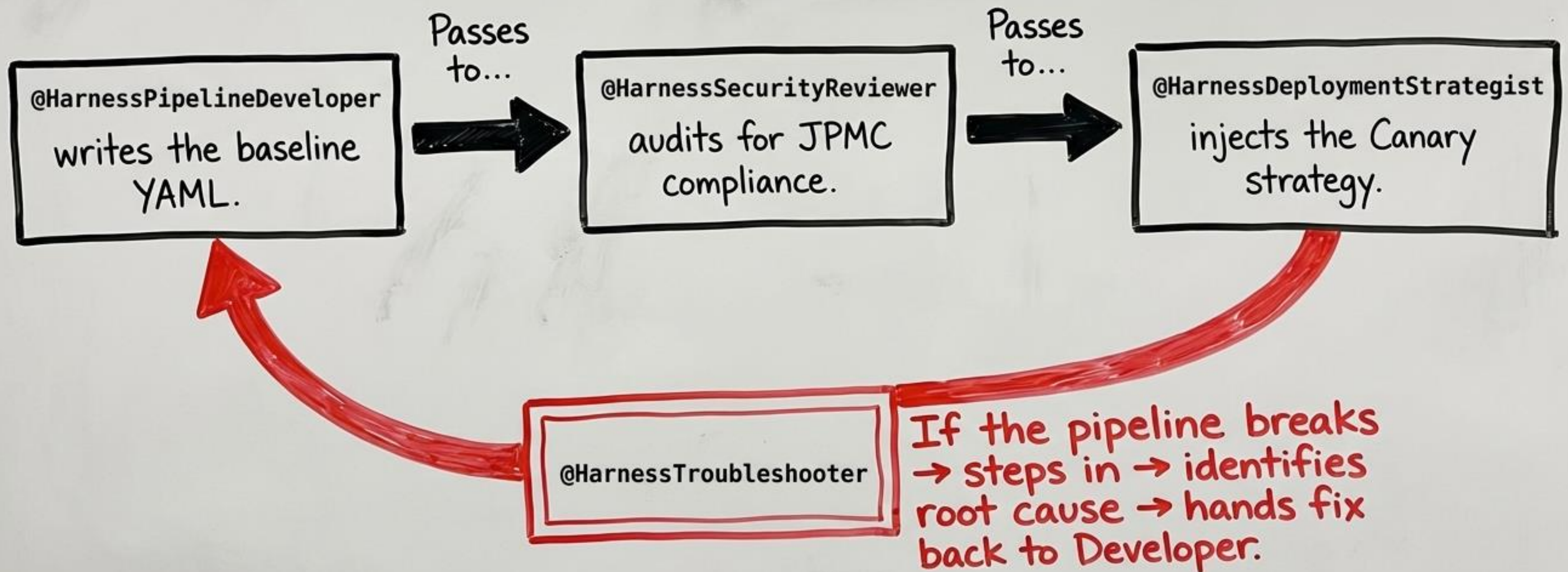
Resource Allocation:

The AI Agent Roster

Available 24/7.
No PTO required.



Cross-Functional Collaboration (Agent Handoffs)



High-Impact Prompts

Syntax: Just type "Use prompt: [name]"

Creation

create-ci-pipeline

Required Inputs: Name,
Module, Build Tool

Deployment

create-ecs-deployment

Required Inputs: Service,
TFE Workspace, Strategy

Security/Gates

add-snow-integration

Required Inputs: Change
Type, Approvers, Env

Fixes

debug-pipeline-failure

Required Inputs:
Error Message

Automated Governance (Zero-Touch Compliance)



No manual
invocation needed.
We're watching.



Phase 1: Pre-Edit Hook

`pre-pipeline-edit.sh`

Validates YAML syntax &
Harness `<+expressions>`
before you can break it.

Phase 2: Auto-Instructions

Activates based on file type.
Enforces context-aware
standards (e.g., Snyk/Trivy
mandatory on `*-ci.yml`).

Phase 3: Post-Edit Hooks

`post-pipeline-edit.sh`
(Scans for hardcoded secrets)
&
`cost-estimate.sh`
(Calculates dollar impact of
your code).

Phase 4: Session End

`collect-session-metrics.sh`

Tracks adoption offline.

The Enterprise Reality (Standardized Red Tape)

ServiceNow

Required for Prod deployments.
Pipeline polls ticket until approved.
Timeout: 1d.

SAML/PCL

Direct IAM is forbidden.
Temporary creds via SAML/PCL
required for all AWS access
(Secrets Manager, ECR, ECS).

MANDATORY

FOR PROD

TFE for ECS

Direct ECS API calls = No.
Must use Terraform Enterprise
(TFE) workspaces for state &
audit trails.

Dynatrace SG

The approved APM for
Continuous Verification.
Mandatory for automated
Prod rollbacks.

Action Item 1: Building the CI Pipeline

Goal: Maven build +
Snyk + ECR publish.

(The Input)

1. Type:

Use prompt: create-ci-pipeline

2. Fill Placeholders:

Name = [Project]-rest-ci

Tool = maven

(The System Magic)

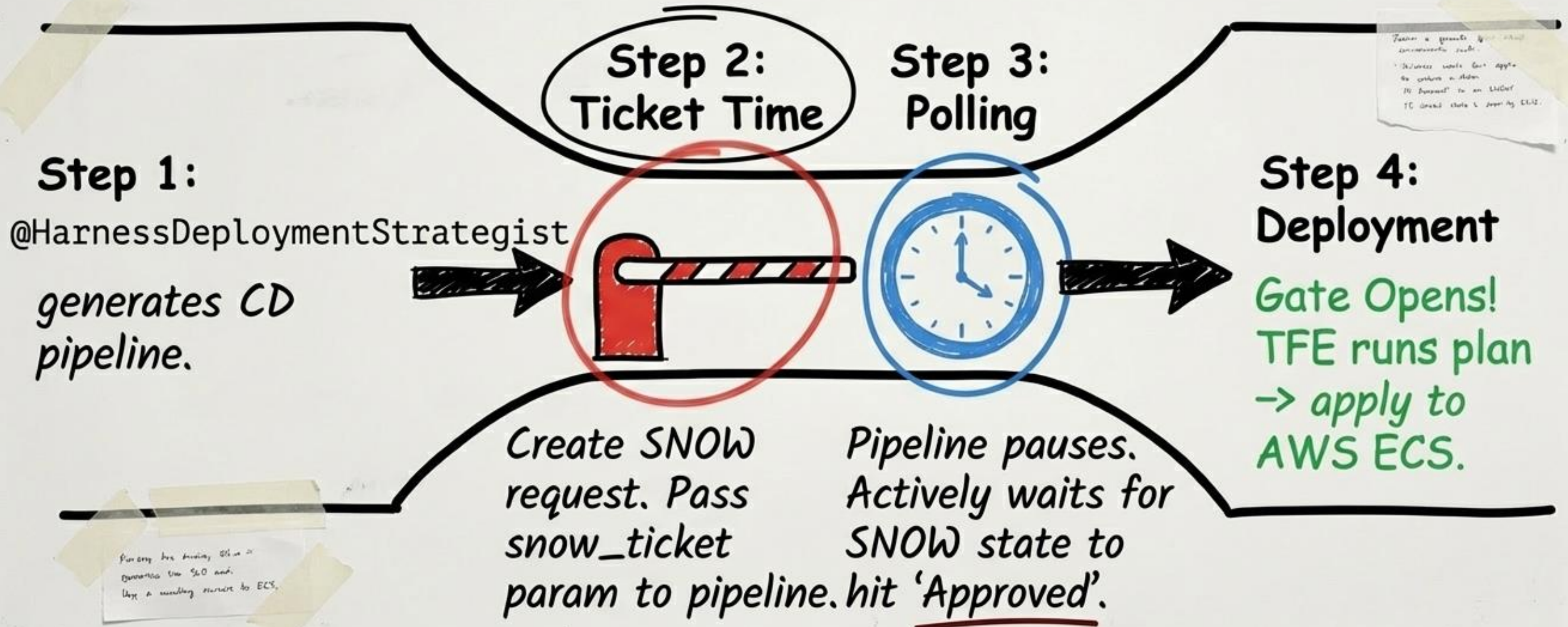
- YAML instantly generated.
- Pre-edit hook ensures syntax is clean.
- Post-edit hook confirms no hardcoded secrets and outputs cost estimate (\$0.05/run).



Result: Compliant
pipeline ready in 3
minutes.

Action Item 2: CD & ServiceNow Gates

Goal: Deploy to Prod
via ECS & TFE.



Action Item 3: Continuous Verification (CV)

Goal: Automated sanity checks with Dynatrace SG.

The Ask: @HarnessMonitoringIntegrator add CV.



Recent engineers
Down to 0.13
latency.
operation number
5-10 mins.



Monitor P95 Latency &
HTTP 5xx Error Rate.

If threshold is breached
during rollout...



...Automated Rollback
via TFE triggers instantly.

Rule of Thumb:
Use 'MEDIUM' sensitivity.
Verification duration: 5-10 mins.



Don't page me at 2 AM.
Let the pipeline roll it back.

Putting out fires (Debugging)

Symptom	Solution
OPA Policy Failed.	Call @HarnessTroubleshooter . Usually missing a required security scan or SNOW approval step.
TFE Workspace Locked.	Previous run failed. Manually force unlock in the TFE UI.
SNOW Timeout.	Check if actual SNOW ticket state matches Harness approval criteria.
YAML Syntax Error.	Pre-edit hook caught it. Fix your 2-space indentation or <code><+pipeline.variables></code> syntax.



Optimizing ROI (Speed & Cost)

Invoke @HarnessPipelineOptimizer



Before ↓

- 15 minute builds.
- All tests run every time.
- Dependencies download sequentially.
- No layer caching.

Each engineer
invokes the SLO
testing
operation, and
5-10 times.

Engineers
Deploy config
using ETF

No competing
for database
created

After (The Fix) ↑

- Enable Cache Intelligence (3 mins → 30s)
- Enable Test Intelligence (8 mins → 2 mins)
- Parallelize Steps & Docker Caching



Impact: 70% time reduction. \$3,650/year saved per pipeline. Synergy achieved.



Let's Circle Back: Action Items

1. Start a session: The `harness-session-start.sh` hook will greet you.
2. Create your first CI: Use prompt: `create-ci-pipeline`
3. Let the agents do the heavy lifting: `@HarnessPipelineDeveloper`

Offline Resources:

- AGENTS.md - Full delegation map
- .github/skills/ - The Knowledge Base

Meeting adjourned. Go write some code (or let Claude do it).